

INSTITUTO FEDERAL DO PARANÁ

JOÃO VITOR GAIOSKI

ÁRVORES E SUAS APLICAÇÕES

IRATI

2026

RESUMO

O seguinte trabalho visa entregar uma introdução ao estudo da estrutura de dados árvore, mostrando seus conceitos, propriedades e demonstrando exemplo práticos junto de figuras didáticas, abrangendo desde conceitos iniciais de sobre nós e caminhos até árvores binárias de busca e suas variantes. O trabalho encontra resultados de uma pesquisa teórica profunda, servindo perfeitamente para iniciar os estudos na área. O conteúdo do trabalho é árvores na computação, suas propriedades e conceitos fundamentais, árvores binárias e seus diversos tipos, árvores binárias de busca e aplicações. Se for de interesse do leitor, nas referências constam exemplos de códigos que implementam o conteúdo aqui comentado.

Palavras-chave: Árvores na computação; Estruturas Não-lineares; Árvores Binárias; Percursos; Árvores Binárias de Busca; aplicações de Árvores.

SUMÁRIO

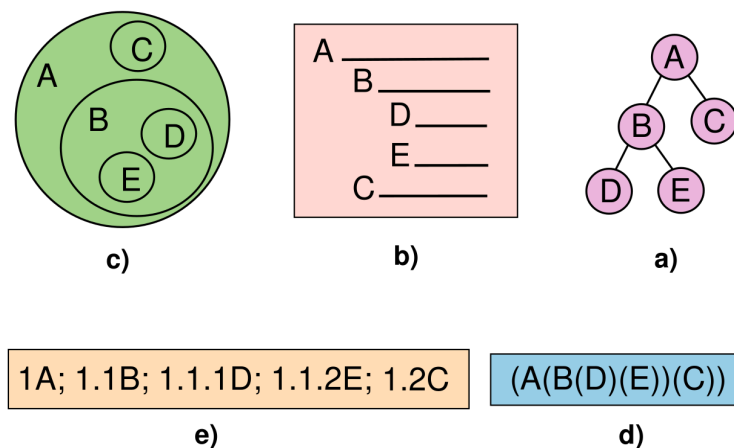
1 INTRODUÇÃO.....	3
2 CONCEITOS FUNDAMENTAIS.....	5
3 ÁRVORES BINÁRIAS.....	7
4 ÁRVORES BINÁRIAS DE BUSCA (BST).....	13
5 CONCLUSÃO.....	18
6 REFERÊNCIAS.....	19

1 INTRODUÇÃO

Na computação, a árvore é uma estrutura de dados não-linear, hierárquica e amplamente utilizada em diversos contextos. Uma das suas principais vantagens é permitir o acesso estruturado, rápido e fácil aos dados que a compõem, diferentemente de estruturas de dados lineares como listas e filas, que armazenam os dados sequencialmente, fazendo com que o tempo para realizar a busca aumente conforme o tamanho dos dados.

Dentre as diversas formas de se representar visualmente uma estrutura de árvore, destacam-se algumas, como a hierárquica, semelhante a um organograma, o diagrama de barras, o diagrama de inclusão, estrutura de aninhamento de parênteses e numeração por níveis.

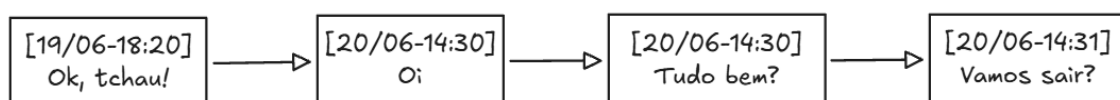
Figura 1 - Diferentes formas de representar uma árvore



Fonte: Dewrys, via Wikimedia Commons¹

Para compreender melhor a diferença de estruturas lineares e não lineares, as lineares possuem uma rigidez: cada elemento está estritamente relacionado sequencialmente com um antecessor e um sucessor, porém, estruturas não-lineares como árvores e grafos podem se relacionar com nenhum ou vários elementos, permitindo representar conexões complexas e operações mais rápidas sobre volumes grandes de dados. Na computação, como exemplos de estruturas lineares bem conhecidas, podemos destacar arrays, listas linkadas, pilhas e filas.

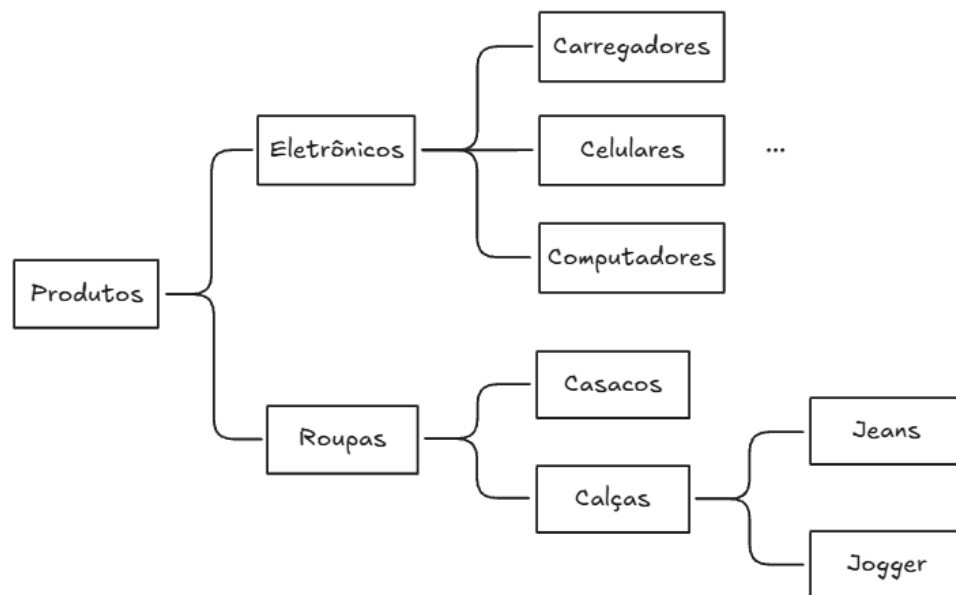
Figura 2 - Mensagens organizadas em uma estrutura linear



Fonte: Elaboração própria

¹ Disponível em <https://commons.wikimedia.org/wiki/File:Tree_structure_representation.svg>. Acesso em 19 de junho de 2026.

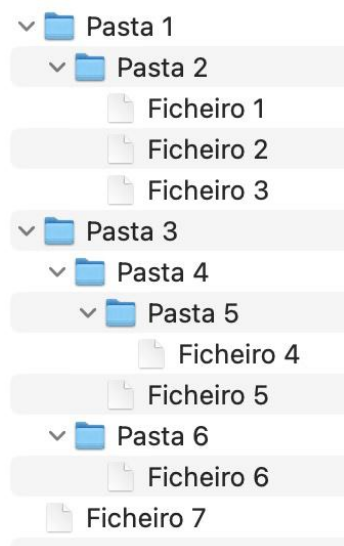
Figura 3 - Produtos organizados em uma estrutura de árvore



Fonte: Elaboração própria

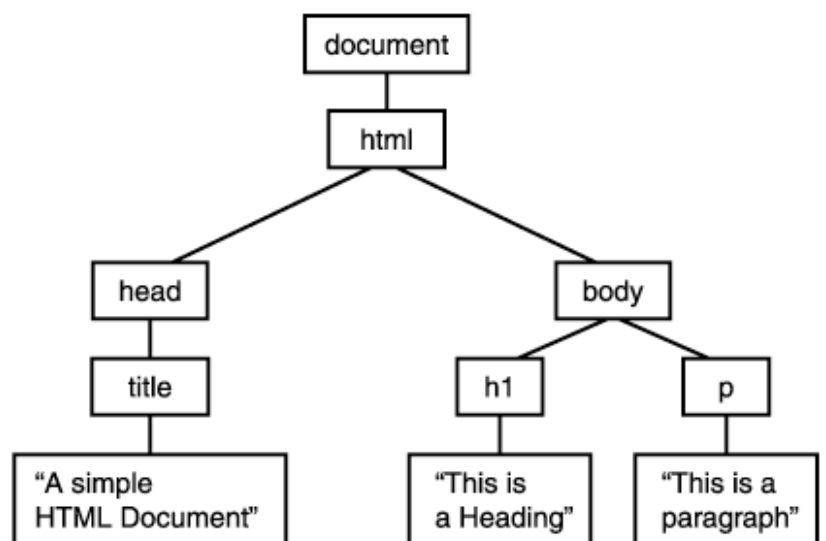
Além do exemplo da Figura 3, diferentes modelos e implementações de árvores são utilizadas em: estrutura de diretórios em um sistema operacional, banco de dados, organização de sites, conexões em redes sociais, entre diversas outras aplicações.

Figura 4 - Diretórios de um SO



Fonte: Alegre123, via Wikimedia Commons²

Figura 5 - HTML simples representado como árvore



Fonte: Isa Weaver, "DOM Tree Structure", via revista Medium³

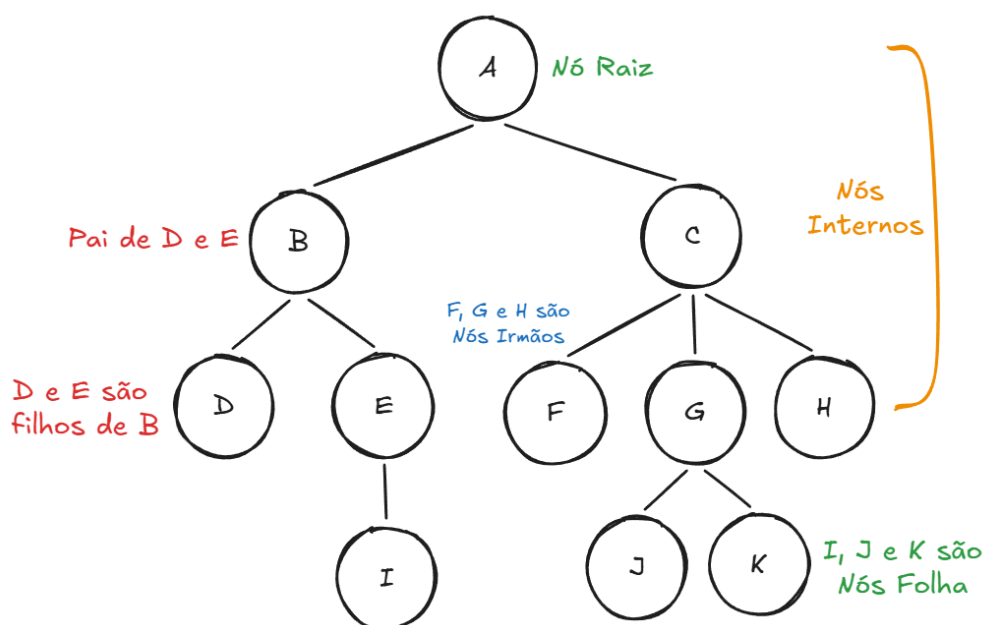
² Disponível em <<https://upload.wikimedia.org/wikipedia/commons/f/f6/Pastas-computador.jpg>>. Acesso em 19 de junho de 2026.

³ Disponível em <<https://medium.datadriveninvestor.com/trees-graphs-b282135e61a6>>. Acesso em 19 de junho de 2026.

2 CONCEITOS FUNDAMENTAIS

Assim como qualquer estrutura de dados, as árvores possuem conceitos fundamentais. Primeiramente, a árvore é composta por **Nós**, cada nó possui um dado que nele é armazenado e pode possuir um conjunto de ponteiros para outros nós, estes que chamamos de **Nós Filhos**. Também é comum chamarmos a ligação entre os nós de **Aresta**. Desta forma, cada nó possui um **Nó Pai**, mas existem exceções: quando o nó é o primeiro do conjunto todo, chamamos ele de **Nó Raiz**, é o nó inicial do arranjo, ele não têm pai; quando o nó é o último do conjunto, ou seja, não possui filhos, este é chamado de **Nó Folha**, nome que faz alusão às folhas de uma árvore, uma vez que representam as extremidades da mesma. Também existe o conceito de **Nó Interno**, todo elemento que possui pelo menos um filho é chamado de nó interno, ou seja, nós folha não entram nesta categoria. Além de pais e filhos, também existem **Nós Irmãos**, que são nós que descendem do mesmo pai.

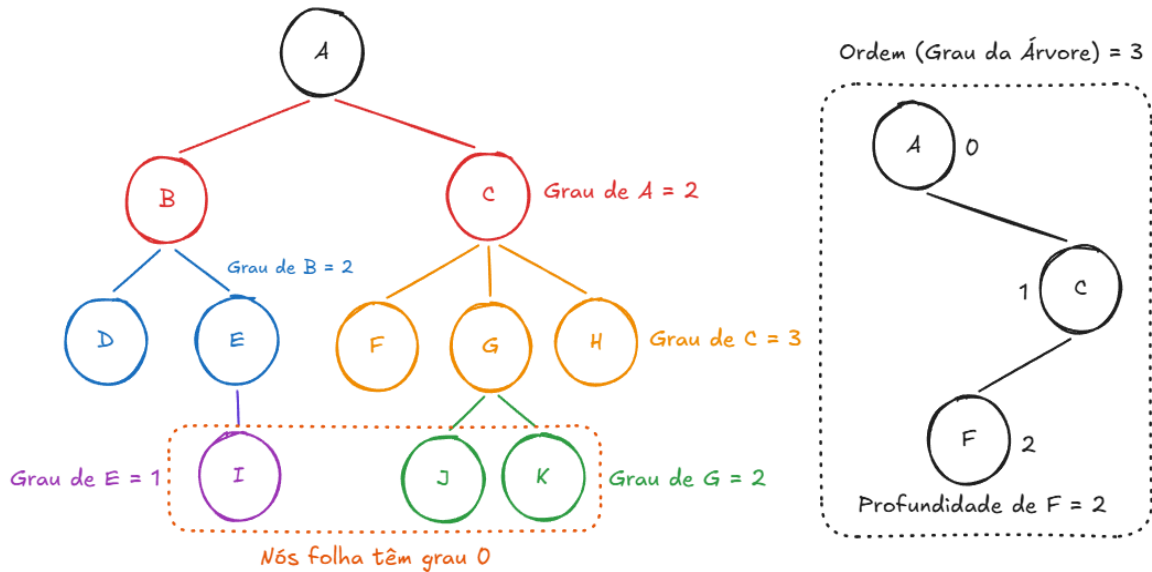
Figura 6 - Exemplificação de conceitos fundamentais de nós



Fonte: Elaboração Própria

Além das classificações de nós, existem conceitos referentes a como vemos os nós em relação ao restante da estrutura. Todo nó possui um **Grau**, este grau é a quantidade de filhos que este nó possui, sendo que nós folha são de grau 0, pois não possuem filhos. Também existe o **Grau da Árvore**, também conhecido como **Ordem**, representado pelo nó de maior grau do conjunto, por exemplo, se o maior grau da árvore for 5, então a ordem é 5. A **Profundidade de um Nó** corresponde à distância deste nó até a raiz, sendo que a raiz tem profundidade 0, por exemplo, se o nó tem profundidade 3, significa que, para navegar até este nó, deve-se atravessar 3 arestas, ou seja, acima dele há 3 nós ancestrais.

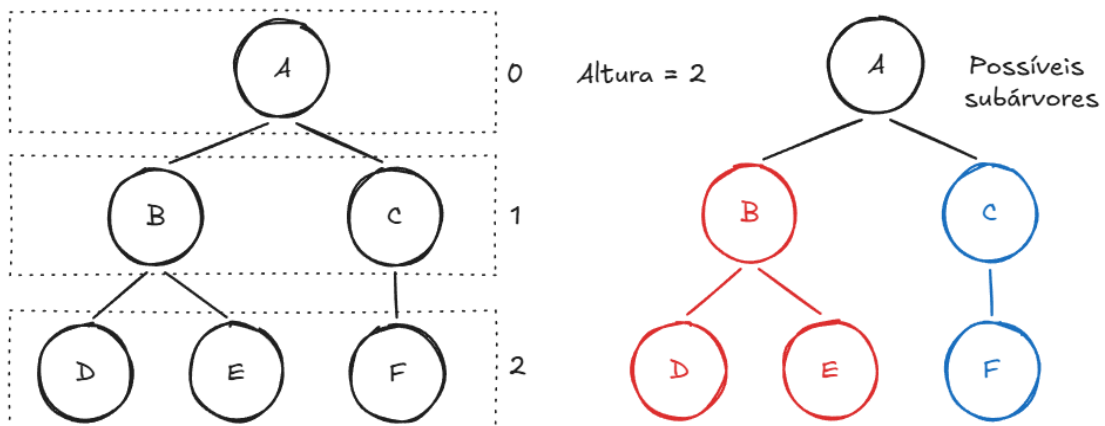
Figura 7 - Exemplificação de Irmãos, Grau, Ordem e Profundidade



Fonte: Elaboração Própria

Para entender melhor sobre profundidade, podemos considerar o conceito de **Caminho**, um caminho nada mais é do que a descrição de quantas unidades, nós ou arestas, você deve percorrer a partir de determinado nó até chegar a outro; por exemplo, para navegar do nó A até C, deve se percorrer um caminho (A, B, C). Uma árvore possui **Níveis**, sendo que cada nó pertence a um nível, começando no nível 0 - que é a raiz -, até o maior nível da estrutura. Desta forma, podemos dizer que o **Nível de um Nó** é o nível de seu pai mais 1. Tendo em mãos o conceito de nível, podemos descrever a **Altura de uma Árvore**, que é a representação do tamanho da mesma, descrita como a distância da raiz até o nó de maior nível. Percebe-se também, que podemos subdividir uma árvore em **Subárvores**, onde, a partir da raiz, cada novo filho representa uma subárvore; é uma forma de dividir a árvore em partes, levando em consideração um nó inicial.

Figura 8 - Altura, Níveis e Subárvores

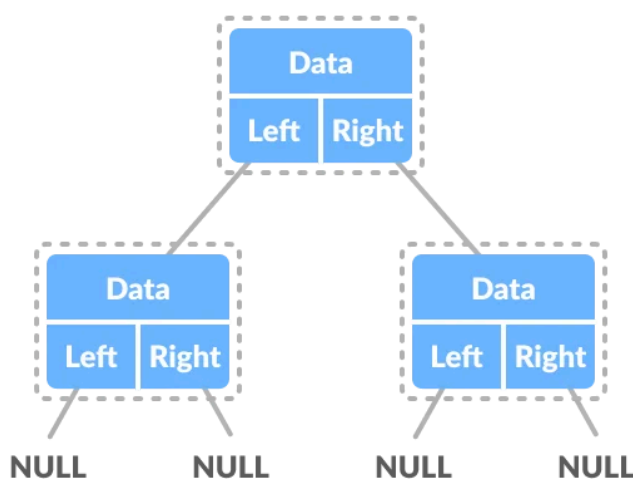


Fonte: Elaboração Própria

3 ÁRVORES BINÁRIAS

Tendo toda essa carga conceitual em mãos, podemos começar a conhecer as **Árvores Binárias**. Inicialmente, precisamos saber que, árvores binárias são aquelas em que os seus nós podem possuir no máximo 2 filhos, e cada nó é composto pelo dado que armazena e por dois endereços, um para a esquerda e outro para a direita, ou seja, para seu filho à esquerda ou para seu filho a direita, como pode ser visto na Figura 9. Esta estrutura possui características únicas que são interessantes serem analisadas. O número máximo de nós em cada nível é 2^L , onde L é o nível em questão; isto se dá pelo fato de que, como se trata de uma árvore binária, ao aumentar o nível, a quantidade de nós dobra. Com isso, temos que, o número total de nós em uma árvore cheia é a soma da quantidade de nós em cada nível, $2^0 + 2^1 + \dots + 2^h$, onde h é a altura da árvore.

Figura 9 - Representação de uma Árvore Binária



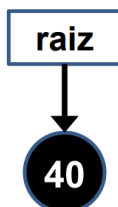
Fonte: Binary Tree, via Programiz⁴

Dentro do estudo de árvores binárias existem diversos tipos possíveis destas. Uma **Árvore Vazia** é simplesmente uma árvore que não possui nenhum nó, tendo nível 0 - ou -1, dependendo da convenção usada - e nenhuma aresta. Uma **Árvore Estritamente Binária** é considerada quando todos os nós da árvore têm exatamente nenhum ou 2 filhos, ou seja, todo elemento têm grau 0 ou 2. Com isso, surgem outros conceitos, como a **Árvore Cheia**, que é uma árvore em que todos os nós internos possuem grau 2, deixando todos os níveis preenchidos. Semelhante, existe a **Árvore Completa**, que além de estar com os níveis totalmente preenchidos, o último nível deve ser preenchido da esquerda para a direita, sendo que o último nó folha não pode possuir um irmão à direita. Também existe a **Árvore Balanceada**, em que, para cada nó, a diferença entre a altura da subárvore à esquerda e a subárvore à direita é 0 ou 1, diferente disto é uma árvore desbalanceada.

⁴ Disponível em <<https://www.programiz.com/dsa/binary-tree>>. Acesso em 24 de junho de 2026.

Figura 10 - Inserção em uma árvore vazia

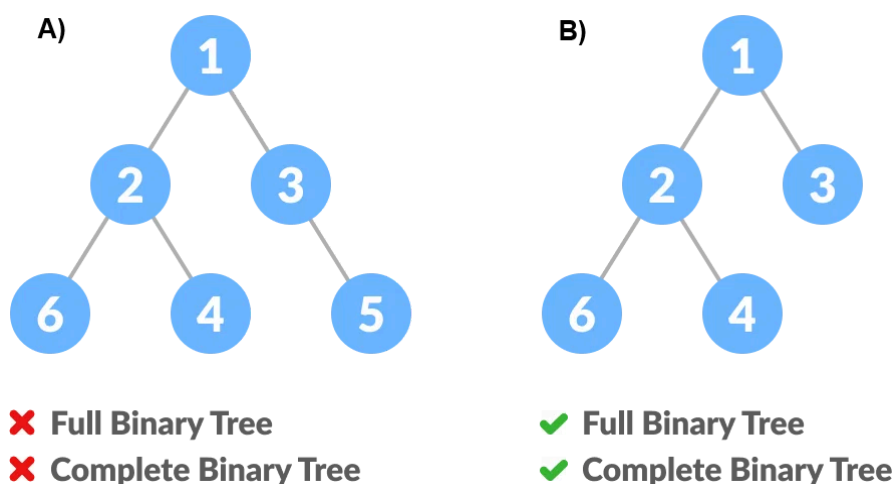
**Inserir o valor '40'
em uma árvore
vazia**



***raiz = novo;**

Fonte: André Backes, "ÁRVORES E ÁRVORE BINÁRIA DE BUSCA"⁵

Figura 11 - (A) Árvore não cheia e não completa. (B) Árvore cheia e completa



Fonte: Binary Tree, via Programiz, modificado⁶

Ao analisar a Figura 11, percebemos que a árvore A não é cheia, pois nem todo nó interno possui grau 2 - o nó 3 possui grau 1 -, e pode-se notar que a árvore também não é completa pois o último elemento, a folha, foi colocada à direita, e a regra diz que deveria ser colocado primeiro à esquerda. Por outro lado, a árvore B é completa e cheia pois atende todos os requisitos.

Cabe então abordar sobre como funciona a inserção e a remoção de elementos na árvore binária. Em quase todos os casos, essas implementações levam em conta a Árvore Binária de Busca (BST), e é o que faremos aqui. Esta estrutura possui características que

⁵ Disponível em <<https://www.facom.ufu.br/~backes/gsi011/Aula10-Arvores.pdf>>, página 24. Acesso em 27 de junho de 2026.

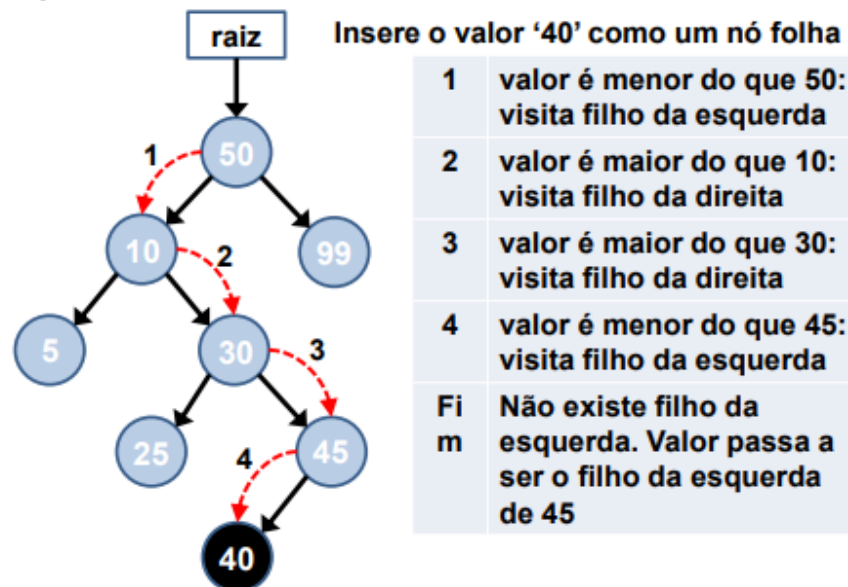
⁶ Disponível em <<https://www.programiz.com/dsa/complete-binary-tree>>. Acesso em 27 de junho de 2026.

permitem realizar operações recursivas, fazendo com que a implementação de inserção e remoção seja simplificada.

A **Inserção** ocorre seguindo esta regra de ordenação: elementos menores ficam à esquerda e elementos maiores à direita. O processo de inserção pode ser analisado como:

1. Compara com a raiz. O valor da raiz é nulo (árvore vazia)? Se sim, insere na raiz, senão, continua;
2. Se o novo valor for menor que o valor do nó atual, caminha à esquerda, se for maior, caminha a direita;
3. Repete este processo até encontrar uma posição vazia, onde um novo nó será acoplado sendo este novo elemento uma folha.

Figura 12 - Etapas de inserção em uma árvore binária



Fonte: André Backes, "ÁRVORES E ÁRVORE BINÁRIA DE BUSCA"⁷

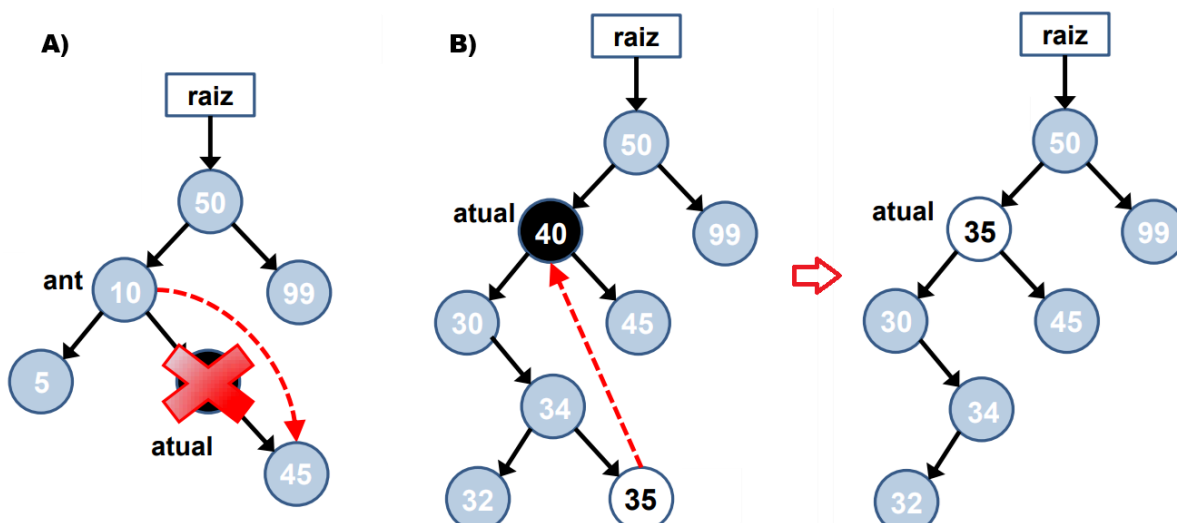
A **Remoção** é consideravelmente mais complexa, pois não podemos deixar buracos na árvore, isto apenas não ocorre caso o elemento a ser removido seja uma folha. Essencialmente, devemos levar 3 casos em consideração:

1. Caso o nó seja uma folha. É o caso mais simples, basta apagar este nó e definir o ponteiro do seu pai para nulo;
2. Caso o nó tenha 1 filho. O filho do elemento removido simplesmente toma o seu lugar, definindo o ponteiro do pai do nó removido para apontar para o filho do nó removido, o "neto". Exemplificação na Figura 13 parte A;
3. Caso o nó tenha 2 filhos. Este caso é mais complicado, pois simplesmente remover o elemento deixa um buraco, então devemos encontrar um substituto. Exemplificação na Figura 13 parte B. A descrição das etapas é:

⁷ Disponível em <<https://www.facom.ufu.br/~backes/gsi011/Aula10-Arvores.pdf>>, página 26. Acesso em 24 de junho de 2026.

- Buscar o menor elemento da subárvore à direita, ou o menor elemento da subárvore à esquerda;
- Substituir o valor do nó a ser removido pelo valor do substituto que encontramos no passo acima;
- Remover o nó substituto de sua posição original. Este passo sempre cai nos casos 1 e 2, ou seja, dividimos o problema em partes menores.

Figura 13 - (A) Remoção de nó com 1 filho. (B) Remoção de nó com 2 filhos



Fonte: André Backes, "ÁRVORES E ÁRVORE BINÁRIA DE BUSCA", modificado⁸

Um assunto que ainda não foi tratado é sobre busca, pois para entender de buscas, precisamos entender de **Percursos** em árvores. Percorrer uma árvore significa passar por todos os seus nós exatamente uma vez, e existem várias formas de fazer isto, sendo que as principais são Pré-ordem, Em-ordem, Pós-ordem e Em Largura. Podemos analisar as minúcias de cada um deles, destacando os termos "ESQUERDA", "RAIZ" e "DIREITA" para melhor compreensão:

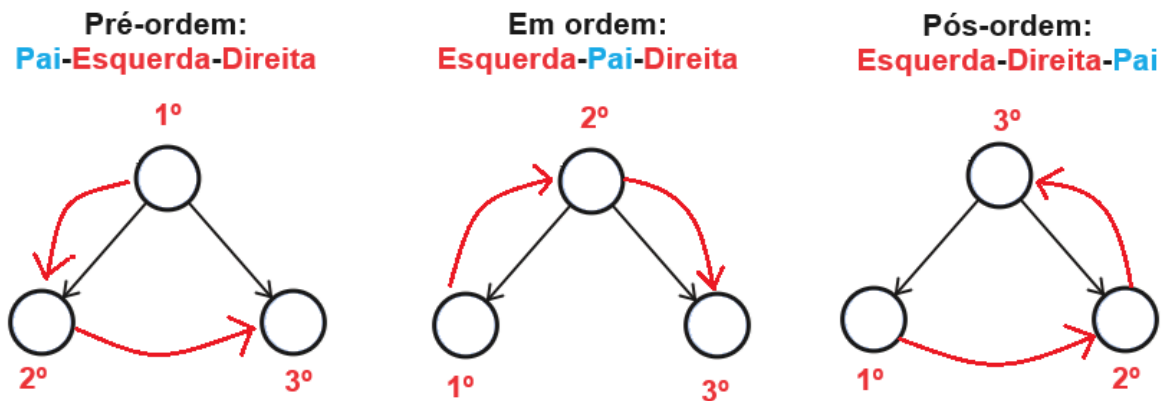
- Percursos **Pré-ordem** primeiro analisam a RAIZ, então a subárvore à ESQUERDA e finalmente a subárvore à DIREITA;
- Percursos **Em-ordem** começam pela subárvore ESQUERDA, passa para a RAIZ e então termina na subárvore DIREITA;
- Percursos **Pós-ordem** iniciam pela subárvore à ESQUERDA, então passa para a subárvore à DIREITA, e finaliza na RAIZ.

Estes três tipos de percursos são considerados por **Profundidade**, a diferença entre eles está na ordem que a raiz é percorrida em relação às subárvores. Também existe o percurso **Em Largura**, onde o percorrimento ocorre em níveis, diferente das buscas em profundidade em que navegamos subárvores inteiras e então passamos para a próxima. Por exemplo, o algoritmo Em Largura analisa primeiro a raiz, então seus filhos, então os seus

⁸ Disponível em <<https://www.facom.ufu.br/~backes/gsi011/Aula10-Arvores.pdf>>, página 31. Acesso em 24 de junho de 2026.

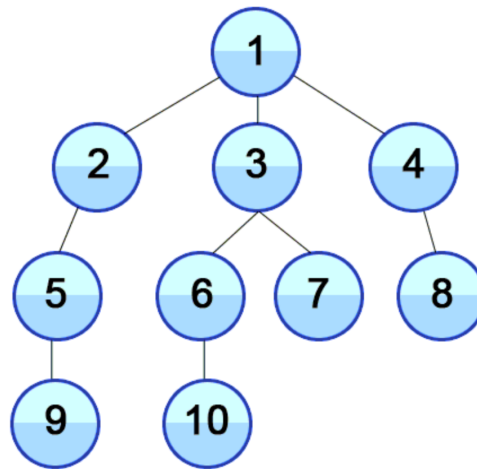
netos, e assim em diante, sempre analisando o nível atual inteiro, da esquerda para a direita e de cima para baixo.

Figura 14 - Simples percursos Pré-ordem, Em-ordem e Pós-ordem



Fonte: Antoniel da Silva Rego, "Percursos em Árvores Binárias para o CNU (TI)", via Estratégia Concursos, modificado⁹

Figura 15 - Percurso em largura onde os nós são a ordem do percorrimto



Fonte: Mre, via Wikimedia Commons¹⁰

Desta forma, é evidente que cada tipo de percurso gera resultados diferentes, isso se dá por conta que cada algoritmo de percorrimto segue uma prioridade temporal diferente de quando um nó é processado. Essencialmente, cada algoritmo destes gera uma sequência de acessos diferente por conta de suas diferentes formas de atuação. Por exemplo, em uma simples estrutura em que temos a raiz A, um filho à esquerda B e um filho à direita C, priorizar o pai (Pré-ordem) gera uma sequência A, B, C; priorizar a esquerda (Em-ordem)

⁹ Disponível em <<https://www.estrategiaconcursos.com.br/blog/percursos-arvores-binarias/>>. Acesso em 26 de junho de 2026.

¹⁰ Disponível em <<https://commons.wikimedia.org/wiki/File:Breadth-First-Search-Algorithm.gif>>. Acesso em 27 de junho de 2026.

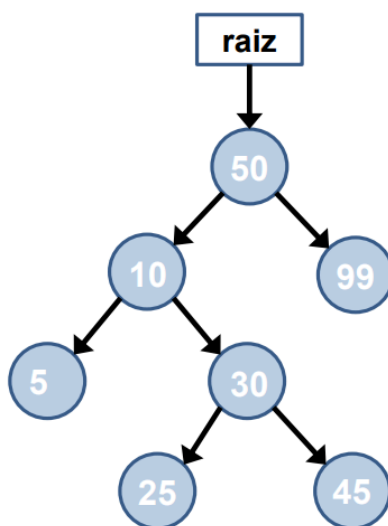
gera uma sequência B, A, C; deixar o pai por último (Pós-ordem) gera uma sequência B, C, A.

Como exemplos de onde cada percurso é adequado: quando deseja-se copiar uma estrutura de árvores - como uma estrutura de pastas, por exemplo -, normalmente usa-se Pré-ordem para garantir que os pais sejam criados antes dos filhos; quando deseja-se obter os dados da estrutura de árvore de forma ordenada - por exemplo, em ordem crescente -, usa-se o algoritmo Em-ordem. O percurso Em-ordem também é usado no algoritmo de Ordenação de Árvores (Tree Sort); quando deseja-se apagar uma estrutura de árvore, normalmente começa-se pelas folhas e subimos até a raiz, pois se deixássemos nós-órfãos isso geraria *memory-leak*, para isso normalmente usa-se o algoritmo Pós-ordem; quando deseja-se encontrar algum dado que sabe-se que está perto da raiz - por exemplo, achar amigos de 1º ou 2º grau em uma rede social -, usa-se o percurso Em Largura, pois neste caso não precisamos necessariamente percorrer todas as subárvores, precisamos percorrer níveis.

4 ÁRVORES BINÁRIAS DE BUSCA (BST)

As **Árvores Binárias de Busca**, ou **BST** (*Binary Search Tree*), são uma implementação de árvores binárias desenvolvidas especialmente para otimizar operações de busca, inserção e remoção de dados. Essencialmente, BSTs possuem duas características fundamentais: todos os elementos da subárvore à esquerda de um nó N são menores do que N , e todos os elementos da subárvore à direita de um nó N são maiores do que N , se existirem. É desta forma que os dados são organizados em árvores binárias de busca, e todas as operações sobre esta árvore seguem essa regra de posicionamento. Perceba na Figura 16 como todos os valores à esquerda de qualquer nó são menores do que o valor deste nó, e os valores à direita são sempre maiores.

Figura 16 - Exemplo de Árvore Binária de Busca



Fonte: André Backes, "ÁRVORES E ÁRVORE BINÁRIA DE BUSCA"¹¹

A ideia base desta estrutura de árvore é o algoritmo de busca binária, também conhecido como "dividir para conquistar", que permite realizar operações rápidas sobre a estrutura, pois a cada comparação, o conjunto todo em que o algoritmo busca pelo elemento reduz pela metade. Este é um algoritmo eficiente, mas seu funcionamento também possui condições:

- Caso a árvore esteja balanceada: busca, inserção e remoção possuem complexidade computacional $O(\log n)$. Este é considerado o caso ideal;
- Caso a árvore esteja desbalanceada (ou degenerada): este é o pior caso, em que a complexidade passa a ser $O(n)$ para as mesmas três funções.

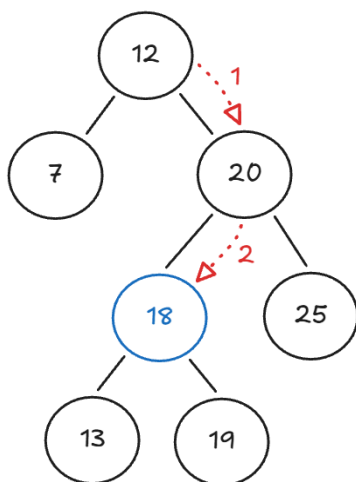
Exemplificando, para uma busca linear encontrar um valor em 1.000.000 registros, ela deve percorrer cada um, ou seja, se o valor desejado estiver no último registro (pior

¹¹ Disponível em <<https://www.facom.ufu.br/~backes/gsi011/Aula10-Arvores.pdf>>, página 22. Acesso em 24 de junho de 2026.

caso), ela terá de fazer 1.000.000 comparações, trata-se de um $O(n)$. Porém, a busca binária têm a vantagem de dividir o problema pela metade a cada ciclo, acelerando muito o processo, sendo que para o mesmo conjunto de 1.000.000 registros, são necessários no máximo 20 passos, um caso $O(\log n)$.

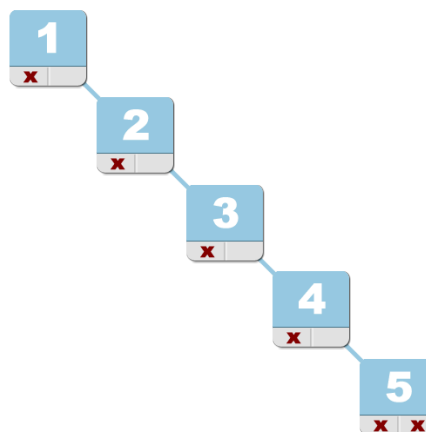
Um exemplo de cenário com o pior caso, ou seja, uma árvore desbalanceada, é quando inserimos dados já ordenados, pois desta forma, ou um nó é sempre maior que seu pai ou é sempre menor, isto faz com que os elementos sejam colocados de forma desbalanceada na árvore, gerando um formato de linha, ou seja, a árvore perde a característica de ramificação e praticamente se torna uma lista encadeada. Por exemplo, se tentarmos inserir uma sequência 1, 2, 3, 4, 5, os elementos sempre são colocados à direita e a árvore tende ao desbalanceamento. Por conta deste último cenário raramente se usa simples BSTs, na prática, costuma-se usar árvores auto-balanceadas como árvores AVL ou rubro-negra.

Figura 17 - Exemplo de busca em Árvore Binária de Busca



Fonte: Elaboração Própria

Figura 18 - Exemplo de árvore degenerada



Fonte: Enzo Ferber, "Linguagem C - Árvores Binárias", via Viva o Linux¹²

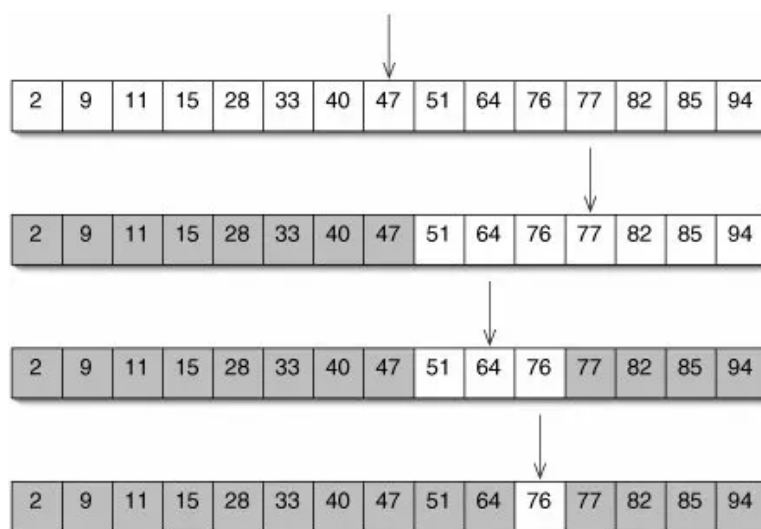
Na Figura 17, percebemos que na primeira comparação, toda a subárvore à esquerda foi deixada de lado, prosseguindo para a direita. Na segunda interação, o mesmo ocorre, eliminando as subárvores à direita e prosseguindo à esquerda, onde finalmente o dado é encontrado. Esta figura representa um exemplo simples, com poucos valores, mas se existisse um grande volume de dados, a grande maioria seria eliminado da busca logo nas primeiras interações, o que mostra a eficácia da busca binária.

Portanto, nada mais justo que abordar aplicações para árvores binárias de busca. Normalmente, esta estrutura é usada quando trabalhamos com grande volume de dados, e quando a aplicação exige busca, inserção e remoção quase simultâneos. Como exemplos:

¹² Disponível em <<https://www.vivaolinux.com.br/artigo/Linguagem-C-Arvores-Binarias/>>. Acesso em 27 de junho de 2026.

dicionários exigem manter uma ordem e exigem buscas completas e parciais, como a busca por uma palavra ou por parte dela com o uso do autocompletar, e a BST é perfeita para implementar uma busca binária neste caso; em um sistema de cadastro, podem existir milhões de registros, porém, BST é ótima para trabalhar com esse volume, tanto para busca, quanto para inserção e para remoção de dados; BSTs também são usadas para indexação de dados, uma vez que permite armazenar chave-valor, garantindo que a busca por um registro ocorra navegando os índices do conjunto ao qual ele pertence e não propriamente ao dado que ele guarda; por fim, sistemas de busca usam BSTs para filtrar os dados usando a estratégia de dividir para conquistar.

Figura 19 - Exemplificação de uma Busca Binária



Fonte: Karuna Sehgal, "A Simplified Interpretation of Binary Search", via revista Medium¹³

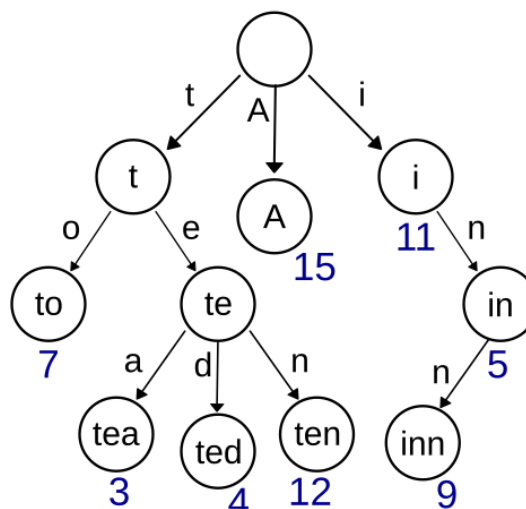
Agora que conhecemos BSTs, vamos nos aprofundar nas aplicações de árvores binárias de busca, podemos analisar alguns cenários e justificar o uso desta estrutura em cada um:

- Existem diversas implementações da ferramenta de **Autocompletar**, porém, o mais comum é utilizar a BST, ou melhor, uma variação conhecida como Árvore de Prefixos (ou *Trie*). Cada nó em uma árvore de prefixos armazena uma única letra, e conforme o algoritmo caminha até as folhas, formam-se palavras agrupando os caracteres. Uma *Trie* é muito eficiente neste caso pois ela prevê possibilidades de palavras conforme o usuário digita; seria inviável usar outra estrutura como array para isto, pois o sistema teria que varrer milhões de strings para encontrar correspondências parciais.

¹³ Disponível em

<<https://medium.com/karuna-sehgal/a-simplified-interpretation-of-binary-search-246433693e0b>>. Acesso em 27 de junho de 2026.

Figura 20 - Exemplificação de Árvore de Prefixos em inglês



Fonte: Booyabazooka, via Wikimedia Commons¹⁴

- **Corretores ortográficos** também se beneficiam de árvores de prefixos e árvores binárias balanceadas, uma vez que seu funcionamento é semelhante ao autocompletar. Todo o conjunto de palavras é carregado na estrutura, permitindo validar instantaneamente se a palavra digitada existe no idioma, e caso ela não exista, sugere opções próximas. Não usa-se tabelas hash, por exemplo, pois elas não permitem a flexibilidade de encontrar semelhantes próximos caso a palavra tenha sido digitada errada, servindo apenas para correspondências exatas.
- **Motores de busca** empregam variações de árvores de busca, conhecidas como Árvores B. Nessas estruturas, a árvore gerencia ponteiros que apontam para o local exato onde as páginas da web ou arquivos estão salvos fisicamente nos servidores. A necessidade de realizar varreduras por intervalos (como buscar páginas modificadas entre duas datas específicas) não é um problema, pois podem ser feitas rapidamente. Optar por uma árvore garante que os dados permaneçam ordenados seguindo uma lógica, porém, tabelas hash seriam ineficientes para consultas de intervalos por não possuírem ordem, enquanto estruturas lineares exigiriam um tempo de processamento não concebível para bases de dados em escalas colossais.
- A lógica em **Sistemas de recomendação** baseia-se em mapear os perfis de usuários e produtos com base em suas preferências e então encontrar os vizinhos mais próximos, aqueles com um perfil semelhante ao do usuário, ou seja, pessoas com gostos muito parecidos, permitindo sugerir filmes, músicas ou produtos de forma personalizada e rápida. A árvore que usamos para isso é uma especialização da BST conhecida como Árvore k-d, um tipo de árvore que funciona com mais de duas dimensões realizando cálculos para encontrar a recomendação ideal, no nosso

¹⁴ Disponível em <https://commons.wikimedia.org/wiki/File:Trie_example.svg>. Acesso em 27 de junho de 2026.

exemplo. Não usamos matrizes ou listas convencionais porque calcular a semelhança de um usuário comparando-o individualmente com outros milhões de cadastros exigiria um esforço computacional inviável; a estrutura de árvore divide o espaço hierarquicamente, permitindo descartar ramos inteiros de perfis incompatíveis logo nos primeiros níveis da busca.

5 CONCLUSÃO

Com este trabalho, aprendemos o que são árvores na computação, árvores binárias, árvores binárias de busca e aplicações, com exemplo e ilustrações para ajudar no entendimento do tema. Conseguimos abstrair os conceitos fundamentais de árvores, as minúcias, as propriedades de árvores binárias, as suas diferentes versões, as suas aplicações e vantagens, e a importância e relevância de árvores binárias de busca e suas variantes.

Pode-se dizer que uma grande dificuldade foi aplicar as normas técnicas para produzir este trabalho acadêmico, uma vez que exige-se padrões bem específicos, mas que com certeza foi ótimo para aprender e aprimorar a escrita de documentos formais. Também leva-se em conta que encontrar as apropriadas imagens e ilustrações para determinados tópicos foi uma das dificuldades encontradas, pois nem sempre é fácil montar um conjunto explicação-ilustração que faça sentido ao leitor.

Finalmente, conclui-se que árvores são extremamente importantes para a computação, fazendo parte da rotina de qualquer pessoa que interaja com um computador, sendo parte fundamental até mesmo do próprio explorador de arquivos do sistema operacional. Percebe-se que existem diversas aplicações para árvores em que elas se sobrepõem sobre outras estruturas de dados por conta de suas características, sendo mais uma escolha no repertório de ferramentas do programador. Claro, a devida estrutura de dados para certa situação nem sempre será uma árvore, isto vai depender do caso, mas essa flexibilidade de árvores é muito interessante para modelar diversos cenários, indo desde armazenar dados de um banco de dados até montar inteligentemente um organograma ou uma árvore genealógica automática para o usuário; existem diversas aplicações em que programar usando a estratégia de árvore pode ser bem interessante e intuitiva.

6 REFERÊNCIAS

- algol.dev/arvores-estrutura-de-dados
- programiz.com/dsa/trees
- inf.ufsc.br/~aldo.vw/estruturas/Arvores/
- normas-abnt.espm.br/index.php?title=Figuras
- [pt.wikipedia.org/wiki/Árvore_\(estrutura_de_dados\)](https://pt.wikipedia.org/wiki/Árvore_(estrutura_de_dados))
- regrasparatecc.com.br/fonte-do-proprio-autor/
- programiz.com/dsa/binary-tree
- programiz.com/dsa/full-binary-tree
- programiz.com/dsa/complete-binary-tree
- geeksforgeeks.org/dsa/complete-binary-tree/
- pt.wikipedia.org/wiki/Árvore_binária
- enjoyalgorithms.com/blog/introduction-to-binary-tree
- facom.ufu.br/~backes/gsi011/Aula10-Arvores.pdf
- estrategiaconcursos.com.br/blog/percursos-arvores-binarias/
- pt.wikipedia.org/wiki/Busca_em_largura
- medium.com/karuna-sehgal/a-simplified-interpretation-of-binary-search-246433693e0b
- freecodecamp.org/portuguese/news/arvores-binarias-de-busca-bst-explicada-com-exemplos/
- en.wikipedia.org/wiki/Trie
- pt.wikipedia.org/wiki/Árvore_B
- pyimagesearch.com/2024/12/23/implementing-approximate-nearest-neighbor-search-with-kd-trees/